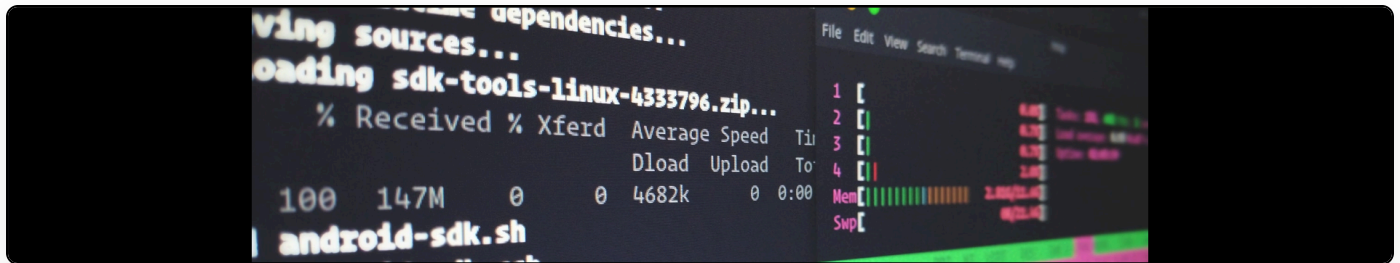


FTP URL FastSnap Parsing in .NET

Cite as: Martin Paul Eve, "FTP URL FastSnap Parsing in .NET", *eve.gd: Martin Paul Eve*, February 01, 2009, <https://doi.org/10.59348/11rxw-mwh37>.



Sometimes, the built in functions of a framework are good enough for your purpose and there is no point in reinventing the wheel. Fine examples of this are to be found at [The Daily WTF](#), one of my personal faves being [The Backup Snippet](#).

However, sometimes the .NET Framework does a poor job of parsing FTP FastSnap URLs. For instance, ftp://ausername:apassword@IP:port/path. This is especially evident when the IP is given in non-dotted decimal representation.

The following function attempts to parse a FastSnap using the inbuilt Framework. If that fails, it tries a custom procedure that is sometimes able to catch some of the failed entries. It's by no means perfect, but it can give some additional flexibility.

```

using System;
using System.Collections.Generic;
using System.Text;
using System.Text.RegularExpressions;

// SNIP: namespace and class declarations

/// <summary>
/// Parses an FTP URL into its components
/// </summary>
/// <param name="fastSnap">The input ftp:// URL</param>
/// <returns>A Dictionary containing host, port, username, password and
path</returns>
public static Dictionary<string, string> GetFastSnap(string fastSnap)
{
    // remove whitespace
    fastSnap = fastSnap.Trim();

    // check for ftp:// at start
    if (!fastSnap.StartsWith("ftp://"))
        fastSnap = string.Format("ftp://{0}", fastSnap);

    // check for non-existent login information
    if (Regex.IsMatch(fastSnap, "^ftp://[^:]*:[0-9]{1,5}$|^ftp://[^:]*:[0-9]{1,5}/.*$"))
        fastSnap = fastSnap.Replace("ftp://",
string.Format("ftp://anonymous:anonymous@{0}", fastSnap));

    // remove surplus double slash
    if (fastSnap.EndsWith("//"))
        fastSnap = fastSnap.TrimEnd('/');

    // check for a trailing slash
    if (!fastSnap.EndsWith("/"))
        fastSnap += "/";

    // try and parse using the in-built Framework function
    try
    {
        Uri ftpUri = new Uri(fastSnap);

        // construct the return object
        Dictionary<string, string> ftpInfoParsed = new Dictionary<string,
string>();

        if (ftpUri.UserInfo != string.Empty)
        {

```

```

        ftpInfoParsed.Add("username", ftpUri.UserInfo.Split(':')[0]);
        ftpInfoParsed.Add("password", ftpUri.UserInfo.Split(':')[1]);
    }
    else
    {
        ftpInfoParsed.Add("username", "anonymous");
        ftpInfoParsed.Add("password", "anonymous");
    }

    ftpInfoParsed.Add("host", ftpUri.Host);
    ftpInfoParsed.Add("port", ftpUri.Port.ToString());
    ftpInfoParsed.Add("path", ftpUri.PathAndQuery);

    return ftpInfoParsed;
}
catch (Exception)
{
}

// ascertain various positions within the string
int firstColon = fastSnap.IndexOf(':', 6);
int atSymbol = fastSnap.IndexOf('@', firstColon);
int secondColon = fastSnap.IndexOf(':', atSymbol);

// non-existent port information
if (secondColon == -1)
{
    secondColon = firstColon + 1;
}

int forwardSlash = fastSnap.IndexOf('/', secondColon);

// construct the return object
Dictionary<string, string> ftpInfo = new Dictionary<string, string>();

// add the fields
ftpInfo.Add("username", fastSnap.Substring(6, firstColon - 6));
ftpInfo.Add("password", fastSnap.Substring(firstColon + 1, atSymbol -
firstColon - 1));

// if no port was specified, we use different offsets
if (secondColon == firstColon + 1)
{
    ftpInfo.Add("host", fastSnap.Substring(atSymbol + 1, forwardSlash -
atSymbol - 1));
    ftpInfo.Add("port", "21");
}
else

```

```
        {
            ftpInfo.Add("host", fastSnap.Substring(atSymbol + 1, secondColon -
atSymbol - 1));
            ftpInfo.Add("port", fastSnap.Substring(secondColon + 1, forwardSlash -
secondColon - 1));
        }

        if (forwardSlash < fastSnap.Length - 1)
        {
            ftpInfo.Add("path", fastSnap.Substring(forwardSlash + 1,
fastSnap.Length - forwardSlash - 1));
        }
        else
        {
            ftpInfo.Add("path", "/");
        }

        return ftpInfo;
    }
}
```